

Competitive Programming

Section 75 Editorial: Master+ Geometry Mixed Contest

Competitive Programming Course

Table of contents

1	How to use this editorial	3
2	A. Convex Lattice Shield	4
2.1	How to find it	4
2.2	Derivation and correctness	4
2.3	Failure-focused tests	4
2.4	C++	4
2.5	Python	5
3	B. Maximum Triangle	7
3.1	How to find it	7
3.2	Derivation and correctness	7
3.3	Failure-focused tests	7
3.4	C++	7
3.5	Python	9
4	C. Robot Collision Translations	10
4.1	How to find it	10
4.2	Derivation and correctness	10
4.3	Failure-focused tests	10
4.4	C++	10
4.5	Python	13
5	D. Weighted Crossings	16
5.1	How to find it	16
5.2	Derivation and correctness	16
5.3	Failure-focused tests	16
5.4	C++	16
5.5	Python	18
6	E. Double-Painted Map	20
6.1	How to find it	20
6.2	Derivation and correctness	20
6.3	Failure-focused tests	20
6.4	C++	20
6.5	Python	22
7	F. Fortress Queries	24
7.1	How to find it	24
7.2	Derivation and correctness	24
7.3	Failure-focused tests	24
7.4	C++	24
7.5	Python	26

1 How to use this editorial

Recover the geometric invariant and degeneracy policy before comparing code.

2 A. Convex Lattice Shield

2.1 How to find it

Neither area nor lattice counting needs interior points. Compress to the hull, then recognize the two independent edge accumulations: cross products for area and gcd steps for boundary points.

2.2 Derivation and correctness

Build the strict hull, apply shoelace, and sum $\gcd(|dx|, |dy|)$ over hull edges. That gcd counts each edge's lattice steps; summing half-open edge counts counts every boundary lattice point exactly once.

2.3 Failure-focused tests

Compare with the independent tiny brute-force oracle. Include collinearity, duplicates, equal event coordinates, boundary contact, and the smallest legal instance.

2.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
struct P {
    int64 x, y;
};
bool operator<(P a, P b) { return tie(a.x, a.y) < tie(b.x, b.y); }
bool operator==(P a, P b) { return a.x == b.x && a.y == b.y; }
P operator+(P a, P b) { return {a.x + b.x, a.y + b.y}; }
P operator-(P a, P b) { return {a.x - b.x, a.y - b.y}; }
int64 cross(P a, P b) { return a.x * b.y - a.y * b.x; }
int64 cross(P a, P b, P c) { return cross(b - a, c - a); }
int64 dist2(P a, P b) {
    int64 x = a.x - b.x, y = a.y - b.y;
    return x * x + y * y;
}
vector<P> hull(vector<P> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
```

```

if (p.size() <= 1)
    return p;
vector<P> h;
for (P x : p) {
    while (h.size() >= 2 && cross(h[h.size() - 2], h.back(), x) <= 0)
        h.pop_back();
    h.push_back(x);
}
size_t lower = h.size();
for (int i = (int)p.size() - 2; i >= 0; --i) {
    P x = p[i];
    while (h.size() > lower && cross(h[h.size() - 2], h.back(), x) <= 0)
        h.pop_back();
    h.push_back(x);
}
h.pop_back();
return h;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n))
        return 0;
    vector<P> p(n);
    for (auto &q : p)
        cin >> q.x >> q.y;
    auto h = hull(p);
    long long area = 0, b = 0;
    for (int i = 0; i < (int)h.size(); ++i) {
        P a = h[i], c = h[(i + 1) % h.size()];
        area += cross(a, c);
        b += gcd(llabs(a.x - c.x), llabs(a.y - c.y));
    }
    cout << llabs(area) << ' ' << b << '\n';
}

```

2.5 Python

```

import sys

def cross(a, b, c=None):
    if c is None:
        return a[0] * b[1] - a[1] * b[0]
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

```

```

def hull(points):
    points = sorted(set(points))
    if len(points) <= 1:
        return points
    lower = []
    for point in points:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], point) <= 0:
            lower.pop()
        lower.append(point)
    upper = []
    for point in reversed(points):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], point) <= 0:
            upper.pop()
        upper.append(point)
    return lower[:-1] + upper[:-1]

import math

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    h = hull(list(zip(data[1::2], data[2::2])))
    area = abs(sum(cross(h[i], h[(i + 1) % len(h)]) for i in range(len(h))))
    b = sum(
        math.gcd(
            abs(h[i][0] - h[(i + 1) % len(h)][0]),
            abs(h[i][1] - h[(i + 1) % len(h)][1]),
        )
        for i in range(len(h))
    )
    print(area, b)

if __name__ == "__main__":
    main()

```

3 B. Maximum Triangle

3.1 How to find it

First move every triangle vertex to the hull without decreasing area. Fix two vertices and inspect area while the third walks around the convex boundary; its unimodality removes one loop.

3.2 Derivation and correctness

A maximum-area triangle uses hull vertices. For fixed ordered i, j , triangle area as k walks forward on a convex polygon is unimodal, so k never moves backward as j advances. The nested scan is $O(h^2)$ rather than cubic.

3.3 Failure-focused tests

Compare with the independent tiny brute-force oracle. Include collinearity, duplicates, equal event coordinates, boundary contact, and the smallest legal instance.

3.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
struct P {
    int64 x, y;
};
bool operator<(P a, P b) { return tie(a.x, a.y) < tie(b.x, b.y); }
bool operator==(P a, P b) { return a.x == b.x && a.y == b.y; }
P operator+(P a, P b) { return {a.x + b.x, a.y + b.y}; }
P operator-(P a, P b) { return {a.x - b.x, a.y - b.y}; }
int64 cross(P a, P b) { return a.x * b.y - a.y * b.x; }
int64 cross(P a, P b, P c) { return cross(b - a, c - a); }
int64 dist2(P a, P b) {
    int64 x = a.x - b.x, y = a.y - b.y;
    return x * x + y * y;
}
vector<P> hull(vector<P> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    if (p.size() <= 1)
```

```

    return p;
vector<P> h;
for (P x : p) {
    while (h.size() >= 2 && cross(h[h.size() - 2], h.back(), x) <= 0)
        h.pop_back();
    h.push_back(x);
}
size_t lower = h.size();
for (int i = (int)p.size() - 2; i >= 0; --i) {
    P x = p[i];
    while (h.size() > lower && cross(h[h.size() - 2], h.back(), x) <= 0)
        h.pop_back();
    h.push_back(x);
}
h.pop_back();
return h;
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n))
        return 0;
    vector<P> p(n);
    for (auto &q : p)
        cin >> q.x >> q.y;
    auto h = hull(p);
    int m = h.size();
    long long ans = 0;
    for (int i = 0; i < m; ++i) {
        int k = i + 2;
        for (int j = i + 1; j < m; ++j) {
            if (k <= j)
                k = j + 1;
            if (k >= m)
                break;
            while (k + 1 < m && llabs(cross(h[i], h[j], h[k + 1])) >
                    llabs(cross(h[i], h[j], h[k])))
                ++k;
            ans = max(ans, llabs(cross(h[i], h[j], h[k])));
        }
    }
    cout << ans << '\n';
}

```

3.5 Python

```
import sys

def cross(a, b, c=None):
    if c is None:
        return a[0] * b[1] - a[1] * b[0]
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

def hull(points):
    points = sorted(set(points))
    if len(points) <= 1:
        return points
    lower = []
    for point in points:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], point) <= 0:
            lower.pop()
        lower.append(point)
    upper = []
    for point in reversed(points):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], point) <= 0:
            upper.pop()
        upper.append(point)
    return lower[:-1] + upper[:-1]

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    h = hull(list(zip(data[1::2], data[2::2])))
    m = len(h)
    ans = 0
    for i in range(m):
        k = i + 2
        for j in range(i + 1, m):
            k = max(k, j + 1)
            if k >= m:
                break
            while k + 1 < m and abs(cross(h[i], h[j], h[k + 1])) > abs(
                cross(h[i], h[j], h[k])
            ):
                k += 1
            ans = max(ans, abs(cross(h[i], h[j], h[k])))
    print(ans)

if __name__ == "__main__":
    main()
```

4 C. Robot Collision Translations

4.1 How to find it

Rewrite intersection algebraically as $\mathbf{a} = \mathbf{b} + \mathbf{t}$. Solving for the query vector gives $\mathbf{t} = \mathbf{a} - \mathbf{b}$, revealing that every valid translation lies in one Minkowski difference.

4.2 Derivation and correctness

A translated B intersects A exactly when there exist $\mathbf{a}$ in A and $\mathbf{b}$ in B with $\mathbf{a} = \mathbf{b} + \mathbf{t}$, equivalently $\mathbf{t} = \mathbf{a} - \mathbf{b}$. Thus valid translations are the closed convex polygon $A + (-B)$. Build it by Minkowski edge merge and answer membership queries logarithmically.

4.3 Failure-focused tests

Compare with the independent tiny brute-force oracle. Include collinearity, duplicates, equal event coordinates, boundary contact, and the smallest legal instance.

4.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
struct P {
    int64 x, y;
};
bool operator<(P a, P b) { return tie(a.x, a.y) < tie(b.x, b.y); }
bool operator==(P a, P b) { return a.x == b.x && a.y == b.y; }
P operator+(P a, P b) { return {a.x + b.x, a.y + b.y}; }
P operator-(P a, P b) { return {a.x - b.x, a.y - b.y}; }
int64 cross(P a, P b) { return a.x * b.y - a.y * b.x; }
int64 cross(P a, P b, P c) { return cross(b - a, c - a); }
int64 dist2(P a, P b) {
    int64 x = a.x - b.x, y = a.y - b.y;
    return x * x + y * y;
}
vector<P> hull(vector<P> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    if (p.size() <= 1)
```

```

    return p;
vector<P> h;
for (P x : p) {
    while (h.size() >= 2 && cross(h[h.size() - 2], h.back(), x) <= 0)
        h.pop_back();
    h.push_back(x);
}
size_t lower = h.size();
for (int i = (int)p.size() - 2; i >= 0; --i) {
    P x = p[i];
    while (h.size() > lower && cross(h[h.size() - 2], h.back(), x) <= 0)
        h.pop_back();
    h.push_back(x);
}
h.pop_back();
return h;
}

vector<P> normalize(vector<P> p) {
    if (p.size() <= 1)
        return p;
    if (cross(p[0], p[1], p[2]) < 0)
        reverse(p.begin(), p.end());
    int at = min_element(p.begin(), p.end(),
        [](P a, P b) { return tie(a.y, a.x) < tie(b.y, b.x); }) -
        p.begin();
    rotate(p.begin(), p.begin() + at, p.end());
    return p;
}

vector<P> minkowski(vector<P> a, vector<P> b) {
    a = normalize(a);
    b = normalize(b);
    if (a.size() < 3 || b.size() < 3) {
        vector<P> s;
        for (P x : a)
            for (P y : b)
                s.push_back(x + y);
        return hull(s);
    }
    int n = a.size(), m = b.size(), i = 0, j = 0;
    vector<P> r = {a[0] + b[0]};
    while (i < n || j < m) {
        P ea = i < n ? a[(i + 1) % n] - a[i] : P{0, 0};
        P eb = j < m ? b[(j + 1) % m] - b[j] : P{0, 0};
        long long z = (i < n && j < m) ? cross(ea, eb) : 0;
        P step;
        if (j == m || (i < n && z > 0))
            step = ea, ++i;
        else if (i == n || z < 0)
            step = eb, ++j;
        else

```

```

        step = ea + eb, ++i, ++j;
        r.push_back(r.back() + step);
    }
    r.pop_back();
    return hull(r);
}

int locate(const vector<P> &p, P q) {
    int n = p.size();
    if (n == 1)
        return q == p[0] ? 0 : -1;
    if (n == 2)
        return cross(p[0], p[1], q) == 0 && min(p[0].x, p[1].x) <= q.x &&
            q.x <= max(p[0].x, p[1].x) && min(p[0].y, p[1].y) <= q.y &&
            q.y <= max(p[0].y, p[1].y)
            ? 0
            : -1;
    long long a = cross(p[0], p[1], q), b = cross(p[0], p.back(), q);
    if (a < 0 || b > 0)
        return -1;
    if (a == 0)
        return dist2(p[0], q) <= dist2(p[0], p[1]) ? 0 : -1;
    if (b == 0)
        return dist2(p[0], q) <= dist2(p[0], p.back()) ? 0 : -1;
    int l = 1, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) / 2;
        if (cross(p[0], p[mid], q) >= 0)
            l = mid;
        else
            r = mid;
    }
    long long z = cross(p[l], p[(l + 1) % n], q);
    return z < 0 ? -1 : (z == 0 ? 0 : 1);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, m, q;
    if (!(cin >> n >> m >> q))
        return 0;
    vector<P> a(n), b(m);
    for (auto &x : a)
        cin >> x.x >> x.y;
    for (auto &x : b)
        cin >> x.x >> x.y;
    reverse(b.begin(), b.end());
    for (auto &x : b)
        x = P{-x.x, -x.y};
    auto s = minkowski(a, b);
    while (q--) {
        P x;

```

```

    cin >> x.x >> x.y;
    cout << (locate(s, x) >= 0 ? "YES\n" : "NO\n");
}
}

```

4.5 Python

```

import sys

def cross(a, b, c=None):
    if c is None:
        return a[0] * b[1] - a[1] * b[0]
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

def hull(points):
    points = sorted(set(points))
    if len(points) <= 1:
        return points
    lower = []
    for point in points:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], point) <= 0:
            lower.pop()
        lower.append(point)
    upper = []
    for point in reversed(points):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], point) <= 0:
            upper.pop()
        upper.append(point)
    return lower[:-1] + upper[:-1]

def normalize(p):
    if len(p) > 2 and cross(p[0], p[1], p[2]) < 0:
        p.reverse()
    at = min(range(len(p)), key=lambda i: (p[i][1], p[i][0]))
    return p[at:] + p[:at]

def minkowski(a, b):
    a = normalize(a)
    b = normalize(b)
    if len(a) < 3 or len(b) < 3:
        return hull([(x[0] + y[0], x[1] + y[1]) for x in a for y in b])
    n, m = len(a), len(b)
    i = j = 0
    r = [(a[0][0] + b[0][0], a[0][1] + b[0][1])]
    while i < n or j < m:

```

```

    ea = (
        (a[(i + 1) % n][0] - a[i][0], a[(i + 1) % n][1] - a[i][1])
        if i < n
        else (0, 0)
    )
    eb = (
        (b[(j + 1) % m][0] - b[j][0], b[(j + 1) % m][1] - b[j][1])
        if j < m
        else (0, 0)
    )
    z = cross(ea, eb) if i < n and j < m else 0
    if j == m or i < n and z > 0:
        step = ea
        i += 1
    elif i == n or z < 0:
        step = eb
        j += 1
    else:
        step = (ea[0] + eb[0], ea[1] + eb[1])
        i += 1
        j += 1
    r.append((r[-1][0] + step[0], r[-1][1] + step[1]))
return hull(r[:-1])

def locate(p, q):
    n = len(p)
    if n == 1:
        return 0 if p[0] == q else -1
    if n == 2:
        return (
            0
            if cross(p[0], p[1], q) == 0
            and min(p[0][0], p[1][0]) <= q[0] <= max(p[0][0], p[1][0])
            and min(p[0][1], p[1][1]) <= q[1] <= max(p[0][1], p[1][1])
            else -1
        )
    a, b = cross(p[0], p[1], q), cross(p[0], p[-1], q)
    if a < 0 or b > 0:
        return -1
    if a == 0:
        return (
            0
            if (q[0] - p[0][0]) ** 2 + (q[1] - p[0][1]) ** 2
            <= (p[1][0] - p[0][0]) ** 2 + (p[1][1] - p[0][1]) ** 2
            else -1
        )
    if b == 0:
        return (
            0
            if (q[0] - p[0][0]) ** 2 + (q[1] - p[0][1]) ** 2

```

```

        <= (p[-1][0] - p[0][0]) ** 2 + (p[-1][1] - p[0][1]) ** 2
        else -1
    )
    l, r = 1, n - 1
    while r - l > 1:
        mid = (l + r) // 2
        if cross(p[0], p[mid], q) >= 0:
            l = mid
        else:
            r = mid
    z = cross(p[l], p[(l + 1) % n], q)
    return -1 if z < 0 else (0 if z == 0 else 1)

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    it = iter(data)
    n, m, q = next(it), next(it), next(it)
    a = [(next(it), next(it)) for _ in range(n)]
    b = [(next(it), next(it)) for _ in range(m)]
    b = [(-x, -y) for x, y in reversed(b)]
    s = minkowski(a, b)
    print(
        "\n".join(
            "YES" if locate(s, (next(it), next(it))) >= 0 else "NO"
            for _ in range(q)
        )
    )

if __name__ == "__main__":
    main()

```

5 D. Weighted Crossings

5.1 How to find it

Start from the unweighted sweep and use linearity: counts become sums of horizontal weights, while each vertical query scales that sum by its own weight.

5.2 Derivation and correctness

The ordinary orthogonal sweep is linear in weights: store the sum of active horizontal weights at each y , query the range sum, and multiply by the vertical weight. Each intersecting pair contributes its product exactly once.

5.3 Failure-focused tests

Compare with the independent tiny brute-force oracle. Include collinearity, duplicates, equal event coordinates, boundary contact, and the smallest legal instance.

5.4 C++

```
#include <bits/stdc++.h>
using namespace std;
struct BIT {
    vector<long long> t;
    BIT(int n) : t(n + 1) {}
    void add(int i, long long v) {
        for (++i; i < (int)t.size(); i += i & -i)
            t[i] += v;
    }
    long long sum(int i) {
        long long s = 0;
        for (; i; i -= i & -i)
            s += t[i];
        return s;
    }
    long long range(int l, int r) { return sum(r) - sum(l); }
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
```

```

int h, v;
if (!(cin >> h >> v))
    return 0;
struct E {
    long long x;
    int type;
    long long y1, y2, w;
};
vector<E> e;
vector<long long> ys;
for (int i = 0; i < h; ++i) {
    long long x1, x2, y, w;
    cin >> x1 >> x2 >> y >> w;
    if (x1 > x2)
        swap(x1, x2);
    e.push_back({x1, 0, y, y, w});
    e.push_back({x2, 2, y, y, -w});
    ys.push_back(y);
}
for (int i = 0; i < v; ++i) {
    long long x, y1, y2, w;
    cin >> x >> y1 >> y2 >> w;
    if (y1 > y2)
        swap(y1, y2);
    e.push_back({x, 1, y1, y2, w});
    ys.push_back(y1);
    ys.push_back(y2);
}
sort(ys.begin(), ys.end());
ys.erase(unique(ys.begin(), ys.end()), ys.end());
sort(e.begin(), e.end(),
    [](E a, E b) { return tie(a.x, a.type) < tie(b.x, b.type); });
BIT bit(ys.size());
long long ans = 0;
for (auto z : e) {
    if (z.type != 1)
        bit.add(lower_bound(ys.begin(), ys.end(), z.y1) - ys.begin(), z.w);
    else {
        int l = lower_bound(ys.begin(), ys.end(), z.y1) - ys.begin(),
            r = upper_bound(ys.begin(), ys.end(), z.y2) - ys.begin();
        ans += z.w * bit.range(l, r);
    }
}
cout << ans << '\n';
}

```

5.5 Python

```
import bisect, sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    h, v = next(it), next(it)
    events = []
    ys = []
    for _ in range(h):
        x1, x2, y, w = next(it), next(it), next(it), next(it)
        x1, x2 = sorted((x1, x2))
        events.extend(((x1, 0, y, y, w), (x2, 2, y, y, -w)))
        ys.append(y)
    for _ in range(v):
        x, y1, y2, w = next(it), next(it), next(it), next(it)
        y1, y2 = sorted((y1, y2))
        events.append((x, 1, y1, y2, w))
        ys.extend((y1, y2))
    ys = sorted(set(ys))
    bit = [0] * (len(ys) + 1)

    def add(i, x):
        i += 1
        while i < len(bit):
            bit[i] += x
            i += i & -i

    def pref(i):
        s = 0
        while i:
            s += bit[i]
            i -= i & -i
        return s

    ans = 0
    for _, kind, y1, y2, w in sorted(events):
        if kind != 1:
            add(bisect.bisect_left(ys, y1), w)
        else:
            ans += w * (
                pref(bisect.bisect_right(ys, y2))
                - pref(bisect.bisect_left(ys, y1))
            )
    print(ans)
```

```
if __name__ == "__main__":  
    main()
```

6 E. Double-Painted Map

6.1 How to find it

Ordinary union length loses how much overlap remains below a covered node. Strengthen the state minimally: retain both once-covered and twice-covered length, then derive how a node's local cover shifts those thresholds.

6.2 Derivation and correctness

Sweep vertical sides as for union area, but each node stores two measures: length covered at least once and length covered at least twice. With local cover at least two, both equal the full interval; with local cover one, twice-covered length equals the children's once-covered length; with local cover zero, both are child sums. This induction makes the root's second measure exact in $O(n \log n)$.

6.3 Failure-focused tests

Compare with the independent tiny brute-force oracle. Include collinearity, duplicates, equal event coordinates, boundary contact, and the smallest legal instance.

6.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using ll = long long;
struct E {
    ll x, y1, y2;
    int d;
};
int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n;
    if (!(cin >> n))
        return 0;
    vector<E> e;
    vector<ll> ys;
    for (int i = 0; i < n; ++i) {
        ll x1, y1, x2, y2;
```

```

cin >> x1 >> y1 >> x2 >> y2;
if (x1 > x2)
    swap(x1, x2);
if (y1 > y2)
    swap(y1, y2);
if (x1 == x2 || y1 == y2)
    continue;
e.push_back({x1, y1, y2, 1});
e.push_back({x2, y1, y2, -1});
ys.push_back(y1);
ys.push_back(y2);
}
if (e.empty()) {
    cout << 0 << '\n';
    return 0;
}
sort(ys.begin(), ys.end());
ys.erase(unique(ys.begin(), ys.end()), ys.end());
sort(e.begin(), e.end(), [](E a, E b) { return a.x < b.x; });
int m = ys.size() - 1;
vector<int> cover(4 * m);
vector<ll> one(4 * m), two(4 * m);
function<void(int, int, int)> pull = [&](int x, int l, int r) {
    ll full = ys[r] - ys[l],
        child1 = r - l == 1 ? 0 : one[2 * x] + one[2 * x + 1],
        child2 = r - l == 1 ? 0 : two[2 * x] + two[2 * x + 1];
    if (cover[x] >= 2)
        one[x] = two[x] = full;
    else if (cover[x] == 1)
        one[x] = full, two[x] = child1;
    else
        one[x] = child1, two[x] = child2;
};
function<void(int, int, int, int, int, int)> upd =
    [&](int x, int l, int r, int ql, int qr, int d) {
        if (qr <= l || r <= ql)
            return;
        if (ql <= l && r <= qr)
            cover[x] += d;
        else {
            int mid = (l + r) / 2;
            upd(2 * x, l, mid, ql, qr, d);
            upd(2 * x + 1, mid, r, ql, qr, d);
        }
        pull(x, l, r);
    };
ll ans = 0, last = e[0].x;
for (int i = 0; i < (int)e.size(); i) {
    ll x = e[i].x;
    ans += (x - last) * two[1];
    while (i < (int)e.size() && e[i].x == x) {

```

```

        int l = lower_bound(ys.begin(), ys.end(), e[i].y1) - ys.begin(),
            r = lower_bound(ys.begin(), ys.end(), e[i].y2) - ys.begin();
        upd(1, 0, m, l, r, e[i].d);
        ++i;
    }
    last = x;
}
cout << ans << '\n';
}

```

6.5 Python

```

import sys

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    if not data:
        return
    it = iter(data)
    n = next(it)
    events = []
    ys = []
    for _ in range(n):
        x1, y1, x2, y2 = next(it), next(it), next(it), next(it)
        x1, x2 = sorted((x1, x2))
        y1, y2 = sorted((y1, y2))
        if x1 != x2 and y1 != y2:
            events.extend(((x1, y1, y2, 1), (x2, y1, y2, -1)))
            ys.extend((y1, y2))
    if not events:
        print(0)
        return
    ys = sorted(set(ys))
    index = {y: i for i, y in enumerate(ys)}
    m = len(ys) - 1
    cover = [0] * (4 * m)
    one = [0] * (4 * m)
    two = [0] * (4 * m)

    def update(x, l, r, ql, qr, d):
        if qr <= l or r <= ql:
            return
        if ql <= l and r <= qr:
            cover[x] += d
        else:
            mid = (l + r) // 2
            update(2 * x, l, mid, ql, qr, d)

```

```

        update(2 * x + 1, mid, r, ql, qr, d)
    full = ys[r] - ys[l]
    child1 = 0 if r - l == 1 else one[2 * x] + one[2 * x + 1]
    child2 = 0 if r - l == 1 else two[2 * x] + two[2 * x + 1]
    if cover[x] >= 2:
        one[x] = two[x] = full
    elif cover[x] == 1:
        one[x], two[x] = full, child1
    else:
        one[x], two[x] = child1, child2

events.sort()
ans = 0
last = events[0][0]
i = 0
while i < len(events):
    x = events[i][0]
    ans += (x - last) * two[1]
    while i < len(events) and events[i][0] == x:
        _, y1, y2, d = events[i]
        update(1, 0, m, index[y1], index[y2], d)
        i += 1
    last = x
print(ans)

if __name__ == "__main__":
    main()

```

7 F. Fortress Queries

7.1 How to find it

Many queries justify preprocessing. A convex polygon viewed from one vertex is an ordered fan of triangles, so angular monotonicity replaces a linear edge scan by binary search.

7.2 Derivation and correctness

The convex hull is the closed region of interest. Rays from `hull[0]` partition it into triangles. The first and last ray reject impossible directions; binary search selects the unique fan wedge, and one cross product distinguishes inside, boundary, and outside.

7.3 Failure-focused tests

Compare with the independent tiny brute-force oracle. Include collinearity, duplicates, equal event coordinates, boundary contact, and the smallest legal instance.

7.4 C++

```
#include <bits/stdc++.h>
using namespace std;
using int64 = long long;
struct P {
    int64 x, y;
};
bool operator<(P a, P b) { return tie(a.x, a.y) < tie(b.x, b.y); }
bool operator==(P a, P b) { return a.x == b.x && a.y == b.y; }
P operator+(P a, P b) { return {a.x + b.x, a.y + b.y}; }
P operator-(P a, P b) { return {a.x - b.x, a.y - b.y}; }
int64 cross(P a, P b) { return a.x * b.y - a.y * b.x; }
int64 cross(P a, P b, P c) { return cross(b - a, c - a); }
int64 dist2(P a, P b) {
    int64 x = a.x - b.x, y = a.y - b.y;
    return x * x + y * y;
}
vector<P> hull(vector<P> p) {
    sort(p.begin(), p.end());
    p.erase(unique(p.begin(), p.end()), p.end());
    if (p.size() <= 1)
```

```

    return p;
vector<P> h;
for (P x : p) {
    while (h.size() >= 2 && cross(h[h.size() - 2], h.back(), x) <= 0)
        h.pop_back();
    h.push_back(x);
}
size_t lower = h.size();
for (int i = (int)p.size() - 2; i >= 0; --i) {
    P x = p[i];
    while (h.size() > lower && cross(h[h.size() - 2], h.back(), x) <= 0)
        h.pop_back();
    h.push_back(x);
}
h.pop_back();
return h;
}

int locate(const vector<P> &p, P q) {
    int n = p.size();
    if (n == 1)
        return q == p[0] ? 0 : -1;
    if (n == 2)
        return cross(p[0], p[1], q) == 0 && min(p[0].x, p[1].x) <= q.x &&
            q.x <= max(p[0].x, p[1].x) && min(p[0].y, p[1].y) <= q.y &&
            q.y <= max(p[0].y, p[1].y)
            ? 0
            : -1;
    long long a = cross(p[0], p[1], q), b = cross(p[0], p.back(), q);
    if (a < 0 || b > 0)
        return -1;
    if (a == 0)
        return dist2(p[0], q) <= dist2(p[0], p[1]) ? 0 : -1;
    if (b == 0)
        return dist2(p[0], q) <= dist2(p[0], p.back()) ? 0 : -1;
    int l = 1, r = n - 1;
    while (r - l > 1) {
        int mid = (l + r) / 2;
        if (cross(p[0], p[mid], q) >= 0)
            l = mid;
        else
            r = mid;
    }
    long long z = cross(p[1], p[(l + 1) % n], q);
    return z < 0 ? -1 : (z == 0 ? 0 : 1);
}

int main() {
    ios::sync_with_stdio(false);
    cin.tie(nullptr);
    int n, q;
    if (!(cin >> n >> q))

```

```

    return 0;
vector<P> p(n);
for (auto &x : p)
    cin >> x.x >> x.y;
auto h = hull(p);
while (q--) {
    P x;
    cin >> x.x >> x.y;
    int z = locate(h, x);
    cout << (z < 0 ? "OUT\n" : z == 0 ? "BOUNDARY\n" : "IN\n");
}
}

```

7.5 Python

```

import sys

def cross(a, b, c=None):
    if c is None:
        return a[0] * b[1] - a[1] * b[0]
    return (b[0] - a[0]) * (c[1] - a[1]) - (b[1] - a[1]) * (c[0] - a[0])

def hull(points):
    points = sorted(set(points))
    if len(points) <= 1:
        return points
    lower = []
    for point in points:
        while len(lower) >= 2 and cross(lower[-2], lower[-1], point) <= 0:
            lower.pop()
        lower.append(point)
    upper = []
    for point in reversed(points):
        while len(upper) >= 2 and cross(upper[-2], upper[-1], point) <= 0:
            upper.pop()
        upper.append(point)
    return lower[:-1] + upper[:-1]

def locate(p, q):
    n = len(p)
    if n == 1:
        return 0 if p[0] == q else -1
    if n == 2:
        return (
            0

```

```

        if cross(p[0], p[1], q) == 0
        and min(p[0][0], p[1][0]) <= q[0] <= max(p[0][0], p[1][0])
        and min(p[0][1], p[1][1]) <= q[1] <= max(p[0][1], p[1][1])
        else -1
    )
a, b = cross(p[0], p[1], q), cross(p[0], p[-1], q)
if a < 0 or b > 0:
    return -1
if a == 0:
    return (
        0
        if (q[0] - p[0][0]) ** 2 + (q[1] - p[0][1]) ** 2
        <= (p[1][0] - p[0][0]) ** 2 + (p[1][1] - p[0][1]) ** 2
        else -1
    )
if b == 0:
    return (
        0
        if (q[0] - p[0][0]) ** 2 + (q[1] - p[0][1]) ** 2
        <= (p[-1][0] - p[0][0]) ** 2 + (p[-1][1] - p[0][1]) ** 2
        else -1
    )
l, r = 1, n - 1
while r - l > 1:
    mid = (l + r) // 2
    if cross(p[0], p[mid], q) >= 0:
        l = mid
    else:
        r = mid
z = cross(p[l], p[(l + 1) % n], q)
return -1 if z < 0 else (0 if z == 0 else 1)

def main():
    data = list(map(int, sys.stdin.buffer.read().split()))
    it = iter(data)
    n, q = next(it), next(it)
    h = hull([(next(it), next(it)) for _ in range(n)])
    out = []
    for _ in range(q):
        z = locate(h, (next(it), next(it)))
        out.append("OUT" if z < 0 else ("BOUNDARY" if z == 0 else "IN"))
    print("\n".join(out))

if __name__ == "__main__":
    main()

```